People`s Democratic Republic of Algeria
The Ministry of Higher Education and Scientific Research

University Of Science And Technology Houari Boumediene
Faculty of Computer Science

Software Testing and Quality Assurance

Software Testing

Fourth year in Computer Science engineering

By: PhD. HAOUARI Ahmed Taha
Info.ahmed.taha@gmail.com

2025/2026

1

Quality Assurance (QA)

Tools and methods to assure quality

Definition

“*Quality assurance is a* ***planned and systematic*** *pattern of all actions necessary to provide adequate* ***confidence*** *that a product or service will* ***satisfy*** *given requirements for quality.***”**
IEEE Std 610.12-1990 (Standard Glossary of Software Engineering Terminology)

“*Quality assurance is part of* ***quality management*** *focused on providing* ***confidence*** *that quality requirements* ***will be fulfilled*****.”**
ISO 9000:2015

“Software quality assurance is a ***set of activitie****s that ensure that* ***software processes and products*** *conform to requirements, standards, and procedures.”*
Roger S. Pressman – *Software Engineering: A Practitioner’s Approach*

“Quality assurance is concerned with ***defining processes and standards*** *that should lead to high-quality software.”*
Ian Sommerville – *Software Engineering*

Quality Assurance vs Quality Control

Feature	Quality Assurance (QA)	Quality Control (QC)
Focus	**Process-Oriented:** Focuses on the methods used to create the product.	**Product-Oriented:** Focuses on the final product or a specific version of it.
Nature	**Proactive:** Aims to prevent defects before they occur.	**Reactive:** Aims to identify and fix defects after they are created.
Goal	To improve development and test processes so that defects do not arise.	To identify (and correct) defects in the finished product.
Activity	Management activity (Defining standards, training, audits).	Testing activity (Executing the software, finding bugs).
Timing	Performed throughout the **entire lifecycle**.	Performed during the **testing phase**.
Example	Creating a checklist for coders to follow.	Running a test script to see if a button works.

SOFTWARE QUALITY MANAGEMENT METHODS: QA VS. QC

QUALITY ASSURANCE (QA) - PROCESS-ORIENTED & PROACTIVE

1 ESTABLISH STANDARDS & PROCESSES
- Define Coding Rules
- Review Guidelines
- Testing Strategies

2 PERFORM REVIEWS & INSPECTIONS
- Peer Reviews
- Code Inspections
- Document Walkthroughs

STATIC TESTING - NO CODE EXECUTION

3 CONDUCT PROCESS AUDITS
- Verify Team Follows Defined Processes
- Identify Process Improvement

4 MONITOR & ANALYZE METRICS
- Bug Trends
- Code Coverage
- Defect Density

RESULTS FEEDBACK LOOP

PRIMARY GOAL:
DEFECT PREVENTION

QUALITY CONTROL (QC) - PRODUCT-ORIENTED & REACTIVE

1 CREATE TEST CASES

Develop Scenarios
Based on Requirements

2 EXECUTE TESTING
- Functional Testing
- Performance Testing
- User Acceptance Testing

DYNAMIC TESTING - RUNNING CODE

3 IDENTIFY & LOG DEFECTS
- Record Bugs
- Issue Reports

IDENTIFY & LOG DEFECTS
- Issue Reports
- Defect Lifecycle

4 RE-TEST & REGRESSION TESTING

Verify Fixes
Ensure No New Bugs Introduced

PRIMARY GOAL:
DEFECT DETECTION

KEY DIFFERENTIATORS

WHEN?
(**QA:** Entire Lifecycle | **QC:** Testing Phase

WHO?
(**QA:** Whole Team | **QC:** Testing Team)

WHAT?
(**QA:** Process & Blueprints | **QC:** Finished Product)

ACTION?
(**QA:** Prevention | **QC:** Correction)

Quality Assurance vs Quality Control

Category	Method	Primary Goal (The "Why")	Action (The "How")
QA	**Process Audits**	To ensure the team is following the development rules.	Checking documentation and workflow logs.
QA	**Code Inspections**	To find logical errors before the code is even run.	A formal, line-by-line review of code by a panel.
QA	**Walkthroughs**	To find design flaws early through team discussion.	An informal meeting where the author explains the logic.
QA	**Standards Setting**	To create a uniform way for everyone to work.	Defining coding rules, file structures, and templates.
QC	**Testing**	To find and document bugs in the software product.	Executing the program to see if it meets requirements.
QC	**Defect Logging**	To track errors so they can be fixed and verified.	Recording bugs in a tracking system (e.g., Jira).

1

Software Testing

Is there a defect !?

Definition

“Software testing is the process of executing a program **with the intent of finding errors**.”
Source: The Art of Software Testing — Glenford J. Myers

“Testing is the process consisting of all life cycle activities, both **static and dynamic**, concerned with planning, preparation, and **evaluation of software products** to determine that they satisfy specified requirements and to demonstrate that they **are fit for purpose**.”
Source: International Software Testing Qualifications Board — ISTQB Glossary

“Testing involves executing a program using artificial data to check that **it behaves as expected**.”
Source: Software Engineering — Ian Sommerville

Testing is considered as a process used to **verify and validate that a software product meets requirements and is fit for use**.
Source: ISO/IEC 25010 — International Organization for Standardization

Definition

In simple:
Software testing is a **systematic and controlled activity within Quality Control (QC)** that involves analyzing and executing software in order to:
•Detect defects,
•Verify compliance with specifications,
•Validate that the system satisfies user needs,
•Provide objective information about software quality.

In this sense, **testing is one of the main operational activities of QC**, where the goal is **not to prove that software is correct**, but to **reveal problems and reduce the risk of failure**.

The objectives of testing

According to the **ISTQB** (International Software Testing Qualifications Board), the objectives of testing are not just about "finding bugs."
Here is a detailed list of the core objectives of testing:

1. Evaluating Work Products
Testing begins long before code is executed. One primary objective is to evaluate requirements, user stories, and design documents.
Goal: To find contradictions or missing information in the "paperwork."
Action: Reviewing a design diagram to ensure it follows the system architecture.

2. Verifying Requirements
This is the "Checklist" objective.
Goal: To confirm that all functional and non-functional requirements have been fulfilled.
Action: If the requirement says "The system must support 100 users," testing proves that it actually does.

The objectives of testing

3. Validating the Test Object
This focuses on the user's perspective.
Goal: To demonstrate that the software is "fit for use" and works as the users and stakeholders expect in their real-world environment.
Action: Checking if the workflow of the app is easy for a student to follow..

4. Building Confidence
Testing provides a "health report" for the software.
Goal: To give management and stakeholders a level of professional certainty about the quality of the software.
Action: Running a suite of "Smoke Tests" to prove the basic features are stable before a big demo.

5. Preventing Defects
This is a proactive objective. By involving testers early in the **Conception Phase**, we stop bugs from ever being coded.
Goal: To reduce the cost of development by finding logic flaws in the requirements.
Action: A tester pointing out that a requirement is impossible to build, saving weeks of wasted coding.

The objectives of testing

6. Finding Failures and Defects
This is the most common objective
Goal: To trigger "failures" (wrong behaviors) so that the underlying "defects" (bugs in the code) can be identified and fixed.
Action: Inputting "9999" into an age field to see if the system crashes.
7. Providing Information for Decision-Making
Testers are like "Information Officers."
Goal: To provide data (test reports) so that stakeholders can make an informed decision: "Is the risk of release low enough to go live?"
Action: Creating a summary report showing that 95% of tests passed, but 5% of security tests failed.
8. Meeting Legal or Regulatory Requirements
In some industries (Medical, Aviation, Finance), testing is a legal requirement.
Goal: To ensure the software adheres to industry-specific standards or government laws.
Action: Testing for data privacy to comply with GDPR or local data laws.

Software Testing Limitations

Software testing is a fundamental activity in software engineering, but it is not perfect. It faces several **technical, human, and organizational limitations** that reduce its effectiveness and make it expensive and difficult to manage.

1. Exhaustive Testing is Impossible

One of the most important limitations is that it is impossible to test all inputs and all execution paths of a program. Even a small program can have **millions or billions of input combinations**

Therefore, testing is always **selective and incomplete**

Some defects will always remain undiscovered.

2. Testing Cannot Guarantee Correctness

Testing can show the presence of defects, but it **cannot prove their absence**.

A program may pass all test cases and still fail in real conditions

This is because only a subset of possible scenarios is tested

Testing reduces risk, but never eliminates it completely.

Software Testing Limitations

3. High Cost and Time Consumption
Testing is often one of the **most expensive phases** in software development. In many projects, testing can consume **30% to 50% of total time and budget.**
This makes testing a major economic constraint in projects.
4. Human Factors and Team Resistance
Testing is not only a technical activity, it is also a **social process**. Developers may feel that testing is a form of criticism of their work.Some teams resist testers because defects are seen as “mistakes”
Communication problems may appear between developers and testers,
Which lead to defensive behavior, lack of cooperation, hiding or ignoring defects
5. Dependence on Test Case Quality
The effectiveness of testing depends strongly on how good the test cases are.
Poorly designed test cases miss important scenarios
Testers may focus only on “normal cases” and ignore edge cases
Lack of experience leads to weak coverage

Software Testing Limitations

6. Incomplete and Changing Requirements
Testing depends on requirements, but:
Requirements are often incomplete, unclear, or ambiguous
In agile systems, requirements change frequently
This leads to constantly updating test cases, thus, instability in the testing process.

7. Difficulty with Complex Systems
Modern software systems are highly complex:
concurrency (multiple processes running at the same time)
distributed systems (cloud, microservices)
external dependencies (APIs, networks, databases)

This makes:reproducing bugs difficult
controlling test environments hard
results sometimes inconsistent

Software Testing Limitations

8. Environment Gap (Test vs Real World)
Software behaves differently in real environments compared to test environments.
different hardware configurations
different user behaviors
unpredictable workloads
Some bugs appear only after deployment (“production bugs”).

9. Regression Overhead
Every modification in software introduces new risks:
existing features must be re-tested
regression testing becomes time-consuming
automation helps, but does not solve everything

Error vs Defect vs Bug vs Failure

1. Error (The Cause): An **Error** is a human action that produces an incorrect result. It happens in the mind or the hands of the person working on the project.
Who: A human (Developer, Analyst, or Designer).
2. Defect / Bug (The Presence)
A **Defect** (also commonly called a **Bug**) is the physical evidence of the error sitting inside the work product (the code or the document).
Where: It lives silently in the code, requirements, or design.
Note: A defect can stay in the software forever and never be found if that specific line of code is never "touched."
3. Failure (The Effect)
A **Failure** is an event where the software does not do what it is supposed to do during execution. This is what the **user** sees.
When: It happens only when the software is running (Dynamic).
Note Every failure is caused by a defect, but not every defect causes a failure (e.g., if the code is never executed).

Error vs Defect vs Bug vs Failure

Term	What is it?	Who/What is involved?	Example
Error	A mistake made by a person.	The Programmer/Analyst.	The builder forgets to put a bolt in the door.
Defect (Bug)	The flaw in the product.	The Code or Document.	The door has a missing bolt (it looks okay).
Failure	The system behaving wrongly.	The User / The Running App.	The door falls off when someone opens it.

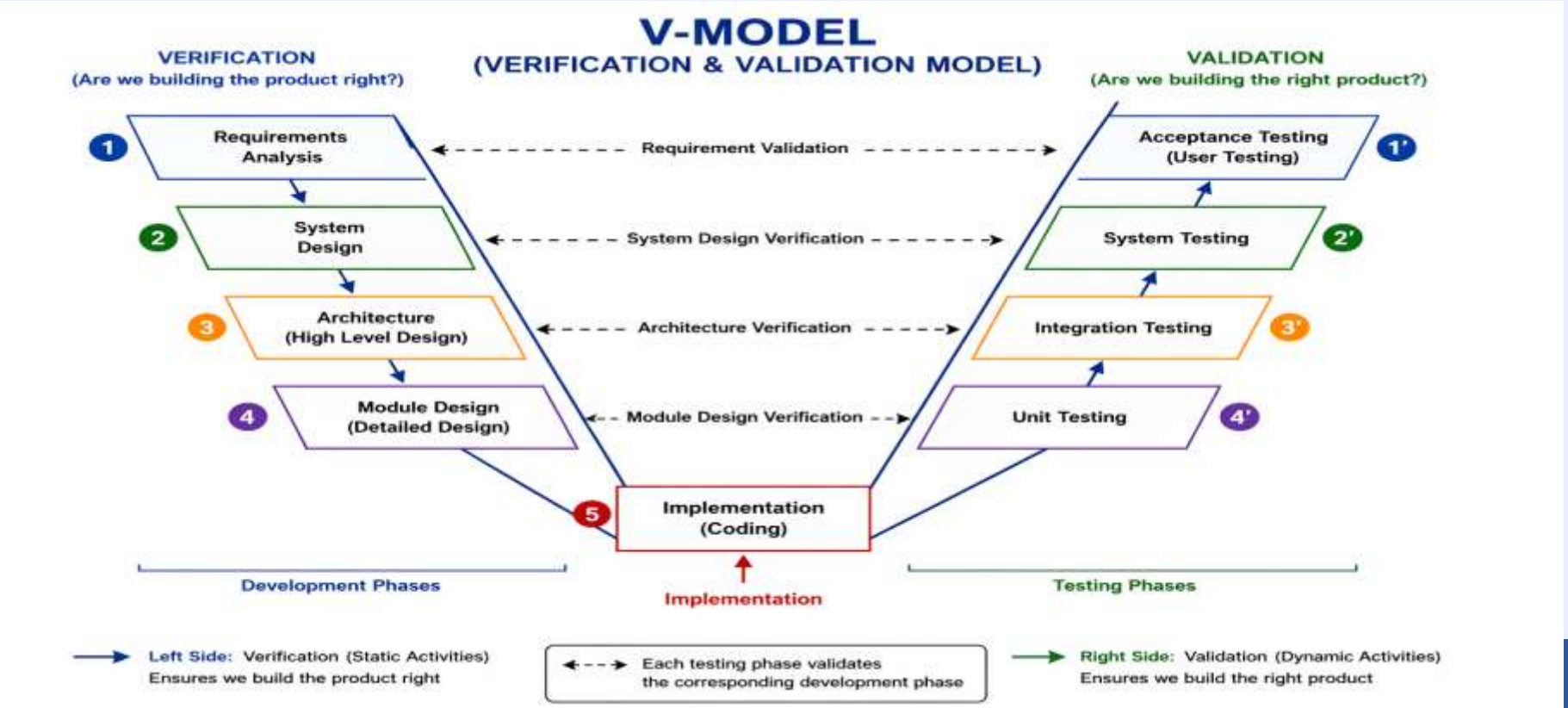
Verification and Validation (V&V)

Verification and Validation (V&V) is a **systematic and continuous set of activities applied throughout the software development life cycle** to ensure software quality.

Verification focuses on checking whether each development work product,such as requirements, design, and code,correctly conforms to its specified requirements, answering the question: ***"Are we building the product correctly?"*** It **is mainly a static process** that does not require program execution and is performed through activities such as reviews, inspections, and formal analysis.

Validation, on the other hand, ensures that the final software system satisfies user needs and intended use, answering the question: ***"Are we building the right product?"*** It is a dynamic process that involves executing the software and evaluating its behavior in real or simulated environments.
Together, verification and validation are not isolated phases but are **integrated at every stage of development,** from requirements to maintenance, ensuring that errors are **detected early and that the final product is both correct with respect to specifications** and useful for its users.

Verification and Validation (V&V)



Verification and Validation (V&V)

EXAMPLE: VERIFICATION vs VALIDATION (Login System)

Requirement: "The password must contain at least 8 characters."

	1) VERIFICATION (Are we building the product correctly?)	2) VALIDATION (Are we building the right product?)
Focus	Requirements, design and implementation conformance to specifications	User needs, real-world usage and satisfaction
Activity	Review requirement and design documents (No execution of the program)	Execute the system and test with real users (Dynamic testing)
What is checked / done	• Requirement says: minimum 8 characters • Design implements this rule • Code follows the design	• Run the login system • Test with real inputs • Observe user behavior and feedback
Example	Requirement / Design Password must contain at least 8 characters. Design	System Execution (Test Case) Enter Password: 12345678 Login User Feedback This is too weak and insecure.
Result	Requirement correctly implemented. Design matches the requirement.	Although the system follows the requirement, users are not satisfied.
Conclusion	System is correct according to the specification. (Verification Successful)	System is not ideal for real user needs. (Validation Failed)

Key Insight: Verification checks whether we built the system correctly according to the specification. Validation checks whether we built the right system for the user.

Testing vs Debugging

Often, testing and debugging are grouped in the same category and are confused with one another. However, the purpose of testing is to **show the presence of errors in a program** and the purpose of debugging is to find errors or misconception that led to the failure of the program and to design and **implement the changes that can correct these errors**. Debugging normally follows testing.

- Testing begins with known conditions, uses pre-defined procedures and has predictable outcomes.
- Debugging starts from possibly unknown initial conditions and the end cannot be predicted
- Testing can be planned, designed and scheduled. However, the procedures and duration of debugging cannot be constrained.
- Testing detects the errors and debugging is a deductive process.
- Testing proves a programmer's failure and debugging a programmers' vindication.
- Testing is subset of verification and validation.
- Testing can be done by an outsider, but debugging must be done by an insider.
- Test execution and design can be automated. Automated debugging is still a vision.

Feature	Testing (Static & Dynamic)	Debugging (Development Activity)
Main Goal	To *demonstrate* that the software fails (find failures).	To *diagnose* and *fix* the bug (the defect).
Simple Question	**"Is the software broken?"**	**"Why is the software broken?"**
When?	Done continuously, but formal phases are throughout development.	Done *after* a failure is reported.
Who does it?	A **Tester** (or sometimes the user).	A **Developer** (the person writing the code).
Process	Dynamic execution of test cases or static review of code/documents.	Deep logical analysis, step-throughs, and code modification.
The Outcome	A **Test Report** showing which tests passed and which failed.	The **Source Code is Changed** and fixed.

Testing vs Debugging

Testing Phase
Test Case
Input: a = 5, b = 3
Expected Output: Sum = 8
Actual Output: Sum = 2
Conclusion (Testing):
The program is incorrect → a defect is detected.

Debugging Phase
Analysis
The result is too small
The operation used is subtraction instead of addition
Bug Identified
Bug fixed : sum=a+b;
New Output: Sum = 8 ✓
The defect is removed

```
#include <stdio.h>
int main() {
int a = 5;
int b = 3;
int sum;
sum = a - b;
printf("Sum = %d\n", sum);
return 0;
}
```

Software Testing Life Cycle (STLC)

The primary goal of software testing is not merely **to find and fix bugs as they appear,**which is the main focus of **debugging**.

Instead, testing is a strategic effort **to prevent errors from ever occurring** in the final product.

By following the structured **Software Testing Life Cycle (STLC)**, quality is integrated from the very beginning of the project.

The software testing process is a **systematic set of activities** performed to plan, design, execute, and evaluate tests in order to ensure software quality. It is not a single step but a **continuous and organized workflow** integrated throughout the development life cycle.

SOFTWARE TESTING LIFE CYCLE (STLC) – EXAMPLE WITH DEFECTS FLOW

Example System: Online Login System

Requirements: 1) Username must not be empty 2) Password must be at least 8 characters 3) User is locked after 3 failed attempts 4) Successful login redirects to dashboard

1 REQUIREMENT ANALYSIS

- Study and understand requirements
- Identify testable requirements
- Identify test conditions and scenarios
- Clarify ambiguities

OUTPUT

- List of testable requirements

2 TEST PLANNING

- Define test strategy
- Define scope
- Estimate effort & time
- Identify resources
- Select tools
- Identify risks

OUTPUT

- Test Plan

3 TEST CASE DESIGN

- Design test cases
- Define test data
- Define expected results
- Review test cases

OUTPUT

- Test Cases
- Test Data

4 TEST ENVIRONMENT SETUP

- Install application
- Configure hardware/ software
- Prepare database
- Verify environment

OUTPUT

- Test Environment Ready

5 TEST EXECUTION

- Execute test cases
- Compare actual vs expected results
- Mark Pass/Fail
- Report defects

OUTPUT

- Test Execution Report
- Defect Reports

6 TEST CLOSURE

- Check completion criteria
- Evaluate test results
- Calculate test coverage
- Document lessons learned
- Prepare summary report

OUTPUT

- Test Summary Report

TEST CASES

TC ID	Scenario	Input (Username / Password)	Expected Result
TC1	Empty username	"" / "password123"	Rejected – show error "Username cannot be empty"
TC2	Short password	"user1" / "1234"	Rejected – show error "Password must be at least 8 characters"
TC3	Valid login	"user1" / "password123"	Login success – redirect to dashboard
TC4	Wrong password	"user1" / "wrongpass"	Show error "Invalid username or password"
TC5	3 failed attempts	3 wrong password attempts	Account locked – show error "Account is locked"
TC6	Login after lock	"user1" / "password123"	Access denied – account still locked

TEST EXECUTION RESULTS

TC ID	Expected Result (From Test Cases)	Actual Result (Observed)	Status
TC1	Rejected – show error	System accepts empty username	✗ Fail
TC2	Rejected – show error	Correctly rejected	✓ Pass
TC3	Login success	Login works correctly	✓ Pass
TC4	Show error message	No error message shown	✗ Fail
TC5	Account locked	Account not locked after 3 attempts	✗ Fail
TC6	Access denied	User can still login after supposed lock	✗ Fail

DEFECTS IDENTIFIED

Defect ID	Related Test Case	Description	Severity	Status
D1	TC1	System accepts empty username	High	Open
D2	TC4	No error message displayed for wrong password	Medium	Open
D3	TC5	Account is not locked after 3 failed attempts	Critical	Open
D4	TC6	User can still log in after account should be locked	Critical	Open

DEFECTS FLOW

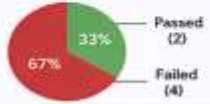
New → Assigned → In Progress → Fixed → Re-test → Closed

If defect is not fixed, it is reopened

TEST CLOSURE SUMMARY

Summary:

- Total Test Cases : 6
- Passed : 2
- Failed : 4
- Defects Found : 4
- Test Coverage : Functional Coverage



Conclusion:

- Multiple critical defects found.
- System is NOT ready for release.
- Fix the critical defects and re-test.

Legend: → Flow of STLC - - → Defects feedback loop

STLC ensures a systematic approach to testing from requirements to closure, with continuous defect tracking and feedback.

Software Testing Life Cycle (STLC)

1. Requirement Analysis

This is the starting point where you identify what needs to be tested based on project goals.

The Goal: Fully understand the test requirements from a quality perspective before writing a single line of code.

Activities

•Study functional and non-functional requirements ,identify testable requirements ,Detect missing or ambiguous requirements, Decide which requirements can be tested

Output

List of testable requirements

2. Test Planning

In this phase, the team defines the high-level strategy, resources, and timeline.

The Goal: Deciding who will perform the tests, what environments (like iPhone or Android) are needed, and how much time the process will take.

Activities Define test strategy ,Estimate time and effort ,Assign resources ,Select tools ,SIdentify risks

Output

Test Plan

Software Testing Life Cycle (STLC)

3. Test Case Development(Test Case Design)
This involves creating the detailed, step-by-step instructions for the actual testing.
The Goal: Prepare the specific data and actions required to verify features we called them **test cases**.
Activities:
Write test cases , Define expected results , Prepare test data ,Review test cases
Output
Test cases ,Test data

4. Test Environment Setup
Before execution, a dedicated "playground" must be prepared where the software can run safely.
The Goal: Configure the necessary hardware, software, and network conditions.
Activities Install the newest version the testing tools systems, Configure hardware/software ,Prepare databases if needed.
Output
Ready test environment

Software Testing Life Cycle (STLC)

5. Test Execution

This is the active phase where you run the prepared tests and record what happens.

The Goal: Compare the actual results against the expected results.

Activities

Execute test cases ,Compare expected vs actual results ,Mark tests as pass/fail ,Report defects

Output

Test execution report ,Defect reports

6. Test Cycle Closure

The final phase involves a wrap-up and reporting of the entire cycle.

The Goal: Analyze results and provide a final sign-off.

Activities

Check if all tests are executed ,Evaluate test coverage ,Document lessons learned ,Prepare final report

Output

Test summary report

Software Testing Life Cycle (STLC)

System Requirements
Username must not be empty
Password must be at least 8 characters
User is locked after **3 failed attempts**
Successful login redirects to dashboard

1. Requirement Analysis
Test Conditions
Empty username → reject
Password < 8 → reject
3 wrong attempts → account locked
Correct credentials → login success

2. Test Planning
Testing type: Functional testing
Method: Manual testing
Tester: 1
Time: 15 min
Objective: Validate login functionality

Software Testing Life Cycle (STLC)

3. Test Case Design

TC ID	Scenario	Input	Expected Result
TC1	Empty username	"" / "password123"	Rejected
TC2	Short password	"user1" / "1234"	Rejected
TC3	Valid login	"user1" / "password123"	Login success
TC4	Wrong password	"user1" / "wrongpass"	Error message
TC5	3 failed attempts	3 wrong logins	Account locked
TC6	Login after lock	Correct credentials	Access denied

Software Testing Life Cycle (STLC)

4 Test Environment Setup

1. Application Setup

Install the login application (web or desktop)

Ensure correct version (e.g., v1.0)

2. Hardware / Platform

PC or laptop

Operating System (e.g., Windows / Linux)

3. Software / Tools

Web browser (Chrome, Firefox)

Testing tool (manual testing

→ no automation tool required)

4. Database Setup

Create test users:

user1 / password123 (valid user)

Locked account scenario (for testing TC5 & TC6)

5. Network Configuration

Ensure server is running

Stable connection between client and server

6. Test Data Preparation

Prepare inputs before execution:

Valid password: password123

Invalid password: 1234

Wrong password: wrongpass

Empty username: ""

Software Testing Life Cycle (STLC)

5. Test Execution

TC ID	Actual Result	Status
TC1	System accepts empty username ✕	Fail
TC2	System correctly rejects short password ✓	Pass
TC3	Login works correctly ✓	Pass
TC4	No error message shown ✕	Fail
TC5	Account not locked after 3 attempts ✕	Fail
TC6	User can still login after supposed lock ✕	Fail

Software Testing Life Cycle (STLC)

6. Test Closure

Defect ID	Test case	Description	Severity	Priority	Status
D1	TC1	System accepts empty username	High	High	Open
D2	TC4	No error message for wrong password	Medium	Medium	Open
D3	TC5	Account not locked after 3 attempts	Critical	High	Open
D4	TC6	User can login after supposed lock	Critical	High	Open

Summary
Total test cases: 6
Passed: 2
Failed: 4
Conclusion
System is **not ready for release**
Critical security defects must be fixed

ing.a.haouari@gmail.com